



# Python Programming Fundamentals

## Working with Data Structures and CRUD — Talk 1: Why Process Data?

Programming Fundamentals Team

SETU

2026-06-23



# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
4. Think–Pair–Share
5. What Is Data?
6. The Problem: One Variable at a Time
7. A Better Solution: A List
8. Common Data Processing Tasks
9. Running Example: Marks
10. Pattern 1: Total
11. Understanding the Accumulator
12. Pattern 2: Average
13. Pattern 3: Count
14. Pattern 4: Search
15. Pattern 5: Highest Value
16. Mini-Program: Mark Report
17. Classroom Activity



# Outline

**18. Why This Matters  
for CRUD**

**19. From Lists to  
Records**

**20. From Records to  
Systems**

**21. Key Takeaways**



# Outline

**1. Why Process Data?**

2. Today's Big Idea

3. Learning Outcomes

4. Think–Pair–Share

5. What Is Data?

6. The Problem: One  
Variable at a Time

7. A Better Solution: A  
List

8. Common Data  
Processing Tasks

9. Running Example:  
Marks

10. Pattern 1: Total

11. Understanding the  
Accumulator

12. Pattern 2: Average

13. Pattern 3: Count

14. Pattern 4: Search

15. Pattern 5: Highest  
Value

16. Mini-Program:  
Mark Report

17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



# 1. Why Process Data?

**Every useful program processes data.**

- Student record systems process marks and attendance
- Online shops process orders and stock
- Games process scores and positions
- AI systems process large datasets



# Outline

1. Why Process Data?

**2. Today's Big Idea**

3. Learning Outcomes

4. Think–Pair–Share

5. What Is Data?

6. The Problem: One Variable at a Time

7. A Better Solution: A List

8. Common Data Processing Tasks

9. Running Example: Marks

10. Pattern 1: Total

11. Understanding the Accumulator

12. Pattern 2: Average

13. Pattern 3: Count

14. Pattern 4: Search

15. Pattern 5: Highest Value

16. Mini-Program: Mark Report

17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways





## 2. Today's Big Idea

Most programs follow this pattern:

**Input → Process → Output**

Example:

student marks → calculate average → mark report



# Outline

1. Why Process Data?

2. Today's Big Idea

**3. Learning Outcomes**

4. Think–Pair–Share

5. What Is Data?

6. The Problem: One Variable at a Time

7. A Better Solution: A List

8. Common Data Processing Tasks

9. Running Example: Marks

10. Pattern 1: Total

11. Understanding the Accumulator

12. Pattern 2: Average

13. Pattern 3: Count

14. Pattern 4: Search

15. Pattern 5: Highest Value

16. Mini-Program: Mark Report

17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



## 3. Learning Outcomes

By the end of this talk you should be able to:

- Explain what data processing means
- Identify common processing tasks
- Understand why lists and dictionaries are useful
- Recognise patterns such as total, count, search and maximum
- Prepare for CRUD-style applications



# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
- 4. Think–Pair–Share**
5. What Is Data?
6. The Problem: One Variable at a Time
7. A Better Solution: A List
8. Common Data Processing Tasks
9. Running Example: Marks
10. Pattern 1: Total
11. Understanding the Accumulator
12. Pattern 2: Average
13. Pattern 3: Count
14. Pattern 4: Search
15. Pattern 5: Highest Value
16. Mini-Program: Mark Report
17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



## 4. Think–Pair–Share

Pick one app you use regularly.

Discuss:

- What data does it store?
- What does it do with that data?
- What output does it show to the user?

Examples: Spotify, TikTok, Netflix, Moodle, banking apps.



# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
4. Think–Pair–Share
- 5. What Is Data?**
6. The Problem: One Variable at a Time
7. A Better Solution: A List
8. Common Data Processing Tasks
9. Running Example: Marks
10. Pattern 1: Total
11. Understanding the Accumulator
12. Pattern 2: Average
13. Pattern 3: Count
14. Pattern 4: Search
15. Pattern 5: Highest Value
16. Mini-Program: Mark Report
17. Classroom Activity





# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



## 5. What Is Data?

Data is information stored in a program.

```
name = "Mary"
```

```
age = 20
```

```
grade = 72
```

This is fine for one student.



# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
4. Think–Pair–Share
5. What Is Data?
- 6. The Problem: One Variable at a Time**
7. A Better Solution: A List
8. Common Data Processing Tasks
9. Running Example: Marks
10. Pattern 1: Total
11. Understanding the Accumulator
12. Pattern 2: Average
13. Pattern 3: Count
14. Pattern 4: Search
15. Pattern 5: Highest Value
16. Mini-Program: Mark Report
17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



## 6. The Problem: One Variable at a Time

What if we have five marks?

grade1 = 72

grade2 = 38

grade3 = 55

grade4 = 90

grade5 = 41

This works, but it does not scale well.



# Outline

1. Why Process Data?

2. Today's Big Idea

3. Learning Outcomes

4. Think–Pair–Share

5. What Is Data?

6. The Problem: One Variable at a Time

**7. A Better Solution: A List**

8. Common Data Processing Tasks

9. Running Example: Marks

10. Pattern 1: Total

11. Understanding the Accumulator

12. Pattern 2: Average

13. Pattern 3: Count

14. Pattern 4: Search

15. Pattern 5: Highest Value

16. Mini-Program: Mark Report

17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



## 7. A Better Solution: A List

```
marks = [72, 38, 55, 90, 41]
```

A list lets us store many values under one name.

This makes it easier to:

- loop through the values
- calculate summaries
- search for values
- update the collection





# Outline

1. Why Process Data?

2. Today's Big Idea

3. Learning Outcomes

4. Think–Pair–Share

5. What Is Data?

6. The Problem: One Variable at a Time

7. A Better Solution: A List

**8. Common Data Processing Tasks**

9. Running Example: Marks

10. Pattern 1: Total

11. Understanding the Accumulator

12. Pattern 2: Average

13. Pattern 3: Count

14. Pattern 4: Search

15. Pattern 5: Highest Value

16. Mini-Program: Mark Report

17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



## 8. Common Data Processing Tasks

When we have a collection of data, we often want to:

- calculate a total
- calculate an average
- count matching items
- search for a value
- find the highest or lowest value
- filter the data



# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
4. Think–Pair–Share
5. What Is Data?
6. The Problem: One Variable at a Time
7. A Better Solution: A List
8. Common Data Processing Tasks
- 9. Running Example: Marks**
10. Pattern 1: Total
11. Understanding the Accumulator
12. Pattern 2: Average
13. Pattern 3: Count
14. Pattern 4: Search
15. Pattern 5: Highest Value
16. Mini-Program: Mark Report
17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



## 9. Running Example: Marks

We will use this dataset:

```
marks = [72, 38, 55, 90, 41]
```

A lecturer might want to know:

- What is the average mark?
- How many students passed?
- What was the highest mark?



# Outline

- |                                        |                                   |                               |
|----------------------------------------|-----------------------------------|-------------------------------|
| 1. Why Process Data?                   | 7. A Better Solution: A List      | 12. Pattern 2: Average        |
| 2. Today's Big Idea                    | 8. Common Data Processing Tasks   | 13. Pattern 3: Count          |
| 3. Learning Outcomes                   | 9. Running Example: Marks         | 14. Pattern 4: Search         |
| 4. Think–Pair–Share                    | <b>10. Pattern 1: Total</b>       | 15. Pattern 5: Highest Value  |
| 5. What Is Data?                       | 11. Understanding the Accumulator | 16. Mini-Program: Mark Report |
| 6. The Problem: One Variable at a Time |                                   | 17. Classroom Activity        |



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways





## 10. Pattern 1: Total

```
marks = [72, 38, 55, 90, 41]
```

```
total = 0
```

```
for mark in marks:  
    total += mark
```

```
print(total)
```

This is an **accumulator** pattern.



# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
4. Think–Pair–Share
5. What Is Data?
6. The Problem: One Variable at a Time
7. A Better Solution: A List
8. Common Data Processing Tasks
9. Running Example: Marks
10. Pattern 1: Total
- 11. Understanding the Accumulator**
12. Pattern 2: Average
13. Pattern 3: Count
14. Pattern 4: Search
15. Pattern 5: Highest Value
16. Mini-Program: Mark Report
17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



# 11. Understanding the Accumulator

At the start:

`total = 0`

Then each mark is added in turn:

$0 + 72 = 72$

$72 + 38 = 110$

$110 + 55 = 165$

$165 + 90 = 255$

$255 + 41 = 296$



# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
4. Think–Pair–Share
5. What Is Data?
6. The Problem: One Variable at a Time
7. A Better Solution: A List
8. Common Data Processing Tasks
9. Running Example: Marks
10. Pattern 1: Total
11. Understanding the Accumulator
12. **Pattern 2: Average**
13. Pattern 3: Count
14. Pattern 4: Search
15. Pattern 5: Highest Value
16. Mini-Program: Mark Report
17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



## 12. Pattern 2: Average

Average needs two pieces of information:

- the total
- the number of items

```
average = total / len(marks)
print(average)
```

`len(marks)` tells us how many items are in the list.



# Outline

1. Why Process Data?

2. Today's Big Idea

3. Learning Outcomes

4. Think–Pair–Share

5. What Is Data?

6. The Problem: One Variable at a Time

7. A Better Solution: A List

8. Common Data Processing Tasks

9. Running Example: Marks

10. Pattern 1: Total

11. Understanding the Accumulator

12. Pattern 2: Average

**13. Pattern 3: Count**

14. Pattern 4: Search

15. Pattern 5: Highest Value

16. Mini-Program: Mark Report

17. Classroom Activity





# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



## 13. Pattern 3: Count

Count the number of passing marks.

```
passes = 0
```

```
for mark in marks:  
    if mark >= 40:  
        passes += 1
```

```
print(passes)
```

This is a **counter** pattern.



# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
4. Think–Pair–Share
5. What Is Data?
6. The Problem: One Variable at a Time
7. A Better Solution: A List
8. Common Data Processing Tasks
9. Running Example: Marks
10. Pattern 1: Total
11. Understanding the Accumulator
12. Pattern 2: Average
13. Pattern 3: Count
- 14. Pattern 4: Search**
15. Pattern 5: Highest Value
16. Mini-Program: Mark Report
17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



## 14. Pattern 4: Search

Check whether a mark of 90 exists.

```
found = False
```

```
for mark in marks:  
    if mark == 90:  
        found = True
```

```
print(found)
```

The variable found remembers whether the search succeeded.



# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
4. Think–Pair–Share
5. What Is Data?
6. The Problem: One Variable at a Time
7. A Better Solution: A List
8. Common Data Processing Tasks
9. Running Example: Marks
10. Pattern 1: Total
11. Understanding the Accumulator
12. Pattern 2: Average
13. Pattern 3: Count
14. Pattern 4: Search
- 15. Pattern 5: Highest Value**
16. Mini-Program: Mark Report
17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



## 15. Pattern 5: Highest Value

```
highest = marks[0]

for mark in marks:
    if mark > highest:
        highest = mark

print(highest)
```

Why do we start with `marks[0]` instead of `0`?





# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
4. Think–Pair–Share
5. What Is Data?
6. The Problem: One Variable at a Time
7. A Better Solution: A List
8. Common Data Processing Tasks
9. Running Example: Marks
10. Pattern 1: Total
11. Understanding the Accumulator
12. Pattern 2: Average
13. Pattern 3: Count
14. Pattern 4: Search
15. Pattern 5: Highest Value
- 16. Mini-Program: Mark Report**
17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



## 16. Mini-Program: Mark Report

```
marks = [72, 38, 55, 90, 41]
```

```
total = 0
```

```
passes = 0
```

```
highest = marks[0]
```

```
for mark in marks:  
    total += mark
```

```
    if mark >= 40:  
        passes += 1
```

```
    if mark > highest:  
        highest = mark
```



## 16. Mini-Program: Mark Report

```
print("Average:", total / len(marks))  
print("Passes:", passes)  
print("Highest:", highest)
```



# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
4. Think–Pair–Share
5. What Is Data?
6. The Problem: One Variable at a Time
7. A Better Solution: A List
8. Common Data Processing Tasks
9. Running Example: Marks
10. Pattern 1: Total
11. Understanding the Accumulator
12. Pattern 2: Average
13. Pattern 3: Count
14. Pattern 4: Search
15. Pattern 5: Highest Value
16. Mini-Program: Mark Report
- 17. Classroom Activity**



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

21. Key Takeaways



## 17. Classroom Activity

Given this data:

```
temperatures = [12, 15, 17, 14, 18, 20, 16]
```

Write down the steps needed to find:

- the highest temperature
- the average temperature
- how many temperatures are above 15



# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
4. Think–Pair–Share
5. What Is Data?
6. The Problem: One Variable at a Time
7. A Better Solution: A List
8. Common Data Processing Tasks
9. Running Example: Marks
10. Pattern 1: Total
11. Understanding the Accumulator
12. Pattern 2: Average
13. Pattern 3: Count
14. Pattern 4: Search
15. Pattern 5: Highest Value
16. Mini-Program: Mark Report
17. Classroom Activity





# Outline

## 18. Why This Matters for CRUD

## 19. From Lists to Records

## 20. From Records to Systems

## 21. Key Takeaways



## 18. Why This Matters for CRUD

CRUD means:

- **Create** data
- **Read** data
- **Update** data
- **Delete** data

Before we can build CRUD applications, we need to be able to process collections of data.



# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
4. Think–Pair–Share
5. What Is Data?
6. The Problem: One Variable at a Time
7. A Better Solution: A List
8. Common Data Processing Tasks
9. Running Example: Marks
10. Pattern 1: Total
11. Understanding the Accumulator
12. Pattern 2: Average
13. Pattern 3: Count
14. Pattern 4: Search
15. Pattern 5: Highest Value
16. Mini-Program: Mark Report
17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

**19. From Lists to  
Records**

20. From Records to  
Systems

21. Key Takeaways



## 19. From Lists to Records

A list of marks stores simple values:

```
marks = [72, 38, 55]
```

A dictionary can store a record:

```
student = {  
    "name": "Mary",  
    "mark": 72  
}
```



# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
4. Think–Pair–Share
5. What Is Data?
6. The Problem: One Variable at a Time
7. A Better Solution: A List
8. Common Data Processing Tasks
9. Running Example: Marks
10. Pattern 1: Total
11. Understanding the Accumulator
12. Pattern 2: Average
13. Pattern 3: Count
14. Pattern 4: Search
15. Pattern 5: Highest Value
16. Mini-Program: Mark Report
17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

**20. From Records to  
Systems**

21. Key Takeaways



## 20. From Records to Systems

A list of dictionaries can store many records:

```
students = [  
    {"name": "Mary", "mark": 72},  
    {"name": "John", "mark": 38},  
    {"name": "Sarah", "mark": 55}  
]
```

This is the starting point for a small Student Manager system.





# Outline

1. Why Process Data?
2. Today's Big Idea
3. Learning Outcomes
4. Think–Pair–Share
5. What Is Data?
6. The Problem: One Variable at a Time
7. A Better Solution: A List
8. Common Data Processing Tasks
9. Running Example: Marks
10. Pattern 1: Total
11. Understanding the Accumulator
12. Pattern 2: Average
13. Pattern 3: Count
14. Pattern 4: Search
15. Pattern 5: Highest Value
16. Mini-Program: Mark Report
17. Classroom Activity



# Outline

18. Why This Matters  
for CRUD

19. From Lists to  
Records

20. From Records to  
Systems

**21. Key Takeaways**



## 21. Key Takeaways

- Data processing is central to programming
- Lists let us store multiple values
- Loops let us process every item
- Common patterns include total, average, count, search and highest value
- These patterns prepare us for CRUD applications



# Thanks for Watching — Any questions?

